Practical : 9	Implementation of Prim's Algorithm (Spanning Tree)
Date : 12/10/2026	

Aim: To implement **Prim's Algorithm in C++** to find the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph.

Objectives :

1. To understand the concept of a Minimum Spanning Tree.
2. To implement Prim's Algorithm using C++.
3. To find the minimum-cost edges required to connect all vertices in a graph.
4. To calculate the total minimum cost of the spanning tree.
5. To analyze the time and space complexity of the algorithm.

Algorithm:

Prim's Algorithm **is a greedy algorithm used to find the Minimum Spanning Tree of a connected, undirected, weighted graph.**

Steps:

1. **Start with a connected, undirected, weighted graph.**
2. **Select any vertex as the starting vertex.**
3. **Mark the starting vertex as visited.**
4. **Find the minimum-weight edge connecting a visited vertex to an unvisited vertex.**
5. **Add that edge to the Minimum Spanning Tree.**
6. **Mark the newly connected vertex as visited.**
7. **Repeat steps 4–6 until all vertices are visited.**
8. **Calculate the total weight of the selected edges.**
9. **Display the Minimum Spanning Tree and its total cost**.

Program:

```
#include <iostream>
#include <climits>
using namespace std;

#define V 5

void primsAlgorithm(int graph[V][V])
{
   int parent[V];
   int key[V];
   bool visited[V];

   for (int i = 0; i < V; i++)
   {
      key[i] = INT_MAX;
      visited[i] = false;
   }

   key[0] = 0;
   parent[0] = -1;

   for (int count = 0; count < V - 1; count++)
   {
      int minKey = INT_MAX;
      int u = -1;

      for (int v = 0; v < V; v++)
      {
         if (!visited[v] && key[v] < minKey)
         {
            minKey = key[v];
            u = v;
         }
      }

      visited[u] = true;

      for (int v = 0; v < V; v++)
      {
         if (graph[u][v] != 0 &&
            !visited[v] &&
            graph[u][v] < key[v])
         {
            parent[v] = u;
```

```
            key[v] = graph[u][v];
        }
    }
}

int totalCost = 0;

cout << "Edge \tWeight\n";

for (int i = 1; i < V; i++)
{
    cout << parent[i] << " - " << i
        << "\t" << graph[i][parent[i]] << endl;

    totalCost += graph[i][parent[i]];
}

cout << "Total Minimum Cost: " << totalCost << endl;
}

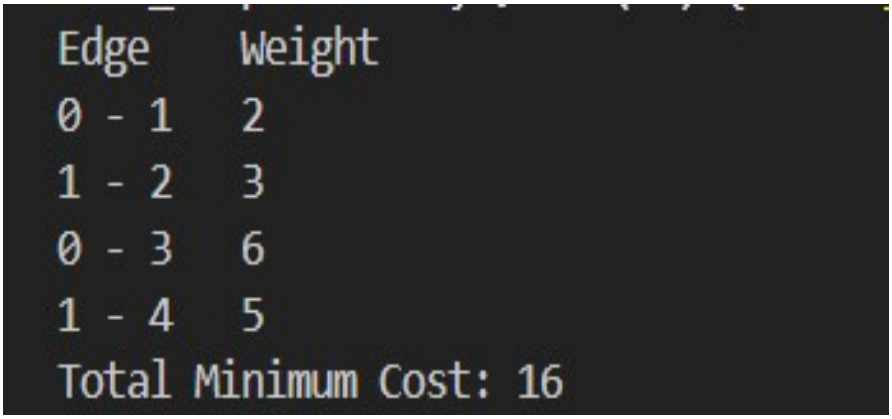
int main()
{
int graph[V][V] = {
    {0, 2, 0, 6, 0},
    {2, 0, 3, 8, 5},
    {0, 3, 0, 0, 7},
    {6, 8, 0, 0, 9},
    {0, 5, 7, 9, 0}
};

primsAlgorithm(graph);

return 0;
}
```

```
            }
        }
    }
    voidDFS(intstart){
        vector<bool>visited(V,false);
        cout <<"\n--- DFS Traversal ---"<<endl;
        DFSUtil(start, visited);
        cout<<endl;
    }
};
intmain(){
    intV,E;
    cout <<"Enter number of vertices: ";
    cin >> V;
    cout <<"Enter number of edges: ";
    cin >> E;
    Graphg(V);
    cout <<"Enter edges (u v):"<<endl; for
    (int i = 0; i < E; i++) {
        intu,v;
        cin>>u>>v;
        if(u<0||u>=V||v<0||v>=V){
            cout<<"Invalidedge.Verticesmustbebetween0and"
                <<V-1<<"."<<endl;
            i--;
            continue;
        }
        g.addEdge(u,v);
    }
    intstart;
    cout <<"Enter starting vertex: ";
    cin >> start;
    if(start<0||start>=V){
        cout <<"Invalid starting vertex."<<endl;
        return 0;
    }
    g.displayGraph();
    g.BFS(start);
    g.DFS(start);
    return 0;
}
```

Output:



Analysis:

The program uses a graph containing five vertices: 0, 1, 2, 3, and 4.

The edge weights are represented using an adjacency matrix.

Vertex 1	Vertex 2	Weight
0	1	2
0	3	6
1	2	3
1	3	8
1	4	5
2	4	7
3	4	9

Working of Prim's Algorithm

The algorithm starts from vertex 0 and selects the minimum-weight edge connecting the current tree to an unvisited vertex.

Step	Selected Edge	Weight	Total Cost
1	0 – 1	2	2
2	1 – 2	3	5
3	1 – 4	5	10
4	0 – 3	6	16

After selecting four edges, all five vertices are connected.

Therefore, the **Minimum Spanning Tree has a total cost of 16**.

Complexity Analysis

Time Complexity: $O(V^2)$

- The algorithm selects the minimum-weight vertex by scanning all vertices.
- It also checks the adjacent vertices using the adjacency matrix.
- These operations result in an overall time complexity of $O(V^2)$.

Space Complexity: $O(V^2)$

- The adjacency matrix requires $O(V^2)$ space.
- The `parent`, `key`, and `visited` arrays require O(V) additional space.
- Therefore, the overall space complexity is $O(V^2)$.

Conclusion :-

Prim's Algorithm was successfully implemented using C++. It finds the Minimum Spanning Tree of a connected, undirected, weighted graph by selecting the minimum-weight edges without forming cycles. The algorithm connects all vertices with the minimum total cost. For the given graph, the minimum spanning tree cost is 16**.**